

Scenario 1: Text to Audio

Request:

```
{  
  "model": "canopylabs/orpheus-v1-english",  
  "input": "[Joy] Every sunrise is a gentle reminder that anything is possible. Make  
today count! Good Morning!",  
  "voice": "austin",  
  "response_format": "wav"  
}
```

The screenshot shows a REST client interface with the following details:

- URL:** `https://api.groq.com/openai/v1/audio/speech`
- Method:** `POST`
- Body:** A JSON object containing the request parameters:

```
{  
  "model": "canopylabs/orpheus-v1-english",  
  "input": "[Joy] Every sunrise is a gentle reminder that anything is possible. Make today count! Good Morning!",  
  "voice": "austin",  
  "response_format": "wav"  
}
```
- Status:** `200 OK` (indicated by a green box)
- Response Time:** `1.34 s`
- Response Size:** `368.21 KB`
- Response Type:** `audio/wav`
- Preview:** A black rectangular area representing the audio waveform, with a playback control bar at the bottom showing `0:07 / 0:07`.

Scenario 2: Multilingual

Request:

```
{
  "model": "llama-3.3-70b-versatile",
  "messages": [
    {
      "role": "system",
      "content": "You are an API Specialist. Always respond in the same language as the user."
    },
    {
      "role": "user",
      "content": "இன்றைய முக்கியமான செய்திகளைப் பற்றி எனக்குச் சொல்லுங்கள்"
    }
  ]
}
```

Response:

```
{
  "id": "chatcmpl-4187f248-1027-408c-bc3b-0ef33c76f427",
  "object": "chat.completion",
  "created": 1781899737,
  "model": "llama-3.3-70b-versatile",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "வணக்கம். இன்றைய முக்கியமான செய்திகள் பற்றி தகவல் தருவதில் மகிழ்ச்சி அடைகிறேன். தற்போதைய செய்திகள் பற்றி அறிந்து கொள்வதற்கு, கீழே கொடுக்கப்பட்டுள்ள தகவல்களைப் பார்க்கவும்.\n\nஅரசியல் செய்திகள்:\nதற்போதைய அரசாங்கம் புதிய கொள்கைகளை அறிவித்துள்ளது. இது நாட்டின் பொருளாதார வளர்ச்சிக்கு உதவும் என்று எதிர்பார்க்கப்படுகிறது.\n\nசுற்றுச்சூழல் செய்திகள்:\nசுற்றுச்சூழல் பாதுகாப்பு தொடர்பான புதிய நடவடிக்கைகள் மேற்கொள்ளப்பட்டு வருகின்றன. இது நாட்டின் சுற்றுச்சூழல் நிலைமைகளை மேம்படுத்துவதற்கு உதவும்.\n\nவிளையாட்டு செய்திகள்:\nதற்போது நடைபெறும் பன்னாட்டு விளையாட்டுப் போட்டிகளில் நம் நாட்டு வீரர்கள் சிறப்பாக செயல்படுகின்றனர். இது நம் நாட்டு மக்களுக்கு பெருமை தருகிறது.\n\nகல்வி செய்திகள்:\nபுதிய கல்வி கொள்கைகள் அறிமுகப்படுத்தப்பட்டுள்ளன. இது மாணவர்களின் கல்வி தரத்தை மேம்படுத்துவதற்கு உதவும்.\n\nதொழில்நுட்ப செய்திகள்:\nபுதிய தொழில்நுட்பங்கள் கண்டுபிடிக்கப்பட்டு வருகின்றன. இது நம் வாழ்க்கையை எளிதாக்குவதற்கு உதவும்.\n\nநாங்கள் இந்த செய்திகளை தொடர்ந்து பார்த்து
```

வருகிறோம். உங்களுக்கு வேறு ஏதேனும் தகவல் தேவைப்பட்டால், எங்களுக்கு தெரிவிக்கவும்."

```
    },
    "logprobs": null,
    "finish_reason": "stop"
  }
],
"usage": {
  "queue_time": 0.174120536,
  "prompt_tokens": 128,
  "prompt_time": 0.006429219,
  "completion_tokens": 1361,
  "completion_time": 2.924272067,
  "total_tokens": 1489,
  "total_time": 2.930701286
},
"usage_breakdown": null,
"system_fingerprint": "fp_45180df409",
"x_groq": {
  "id": "req_01kvgr0b0ye2dbvmatdk1v6j2j",
  "seed": 2017462700
},
"service_tier": "on_demand"
}
```

Scenario 3: SwaggerHub code – Test case

Request

```
{
  "model": "openai/gpt-oss-120b",
  "messages": [
    {
      "role": "system",
      "content": "You are a Senior API Test Automation Architect with strong expertise in Java, Rest Assured, TestNG, Maven, and API schema-driven test generation."
    },
    {
      "role": "user",
      "content": "I – INSTRUCTION\nGenerate Rest Assured Java code for Swagger Petstore APIs.\n\nMANDATORY RULES:\n1) Generate ONLY Java Rest Assured code.\n2) Do NOT include explanations, markdown, or extra text.\n3) Use TestNG as the test runner.\n4) Use meaningful class, method, and variable names.\n5) Include imports required for Rest Assured, TestNG, and assertions.\n6) Use reusable base URI setup.\n7) Cover CRUD operations for Pet resource.\n8) Add assertions for status code and important response fields.\n9) Keep the code clean, executable, and production-style.\n10) Do NOT use placeholder comments.\n\nCONTEXT\nThe code will be used by
```

QA engineers for API automation practice using Swagger Petstore.\n\nP – PERSONA\nYou are a Senior API Test Automation Architect creating enterprise-style reusable Rest Assured automation code.\n\n0 – OUTPUT\nReturn ONLY executable Java code with:\n- One test class\n- @BeforeClass setup\n- Test methods for createPet, getPetById, updatePet, deletePet\n- Request body creation\n- Response validation\n\nRules:\n- No markdown fences\n- No explanation text\n- No assumptions outside Swagger Petstore Pet API\n- Use business-readable method names\n\nTARGET API:\nSwagger Petstore\nResource: Pet\nOperations required:\n- Add a new pet\n- Find pet by ID\n- Update an existing pet\n- Delete a pet"

```
}
]
}
```

Response

```
{
  "id": "chatcmpl-002cbb46-5821-4add-94bc-e3a940b603c7",
  "object": "chat.completion",
  "created": 1781899948,
  "model": "openai/gpt-oss-120b",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "import io.restassured.RestAssured;\nimport
io.restassured.http.ContentType;\nimport io.restassured.response.Response;\nimport
org.testng.Assert;\nimport org.testng.annotations.BeforeClass;\nimport
org.testng.annotations.Test;\n\nimport java.util.ArrayList;\nimport java.util.HashMap;\nimport
java.util.List;\nimport java.util.Map;\n\npublic class PetApiCrudTest {\n\n    private static long
petId;\n\n    @BeforeClass\n    public void setUp() {\n        RestAssured.baseURI =
\"https://petstore.swagger.io/v2\";\n    }\n\n    @Test\n    public void createPet() {\n        petId =
System.currentTimeMillis();\n        Map<String, Object> category = new HashMap<>();\n
category.put(\"id\", 1);\n        category.put(\"name\", \"Dogs\");\n\n        List<String> photoUrls
= new ArrayList<>();\n        photoUrls.add(\"http://example.com/photo1.jpg\");\n\n
Map<String, Object> tag = new HashMap<>();\n        tag.put(\"id\", 1);\n        tag.put(\"name\",
```

```

    \friendly\");\n    List<Map<String, Object>> tags = new ArrayList<>();\n
    tags.add(tag);\n\n    Map<String, Object> petBody = new HashMap<>();\n
    petBody.put(\id, petId);\n    petBody.put(category, category);\n
    petBody.put(name, Rex);\n    petBody.put(photoUrls, photoUrls);\n
    petBody.put(tags, tags);\n    petBody.put(status, available);\n\n    Response
    response = RestAssured\n        .given()\n        .contentType(ContentType.JSON)\n
    .body(petBody)\n        .when()\n        .post(/pet)\n        .then()\n
    .extract()\n        .response();\n\n    Assert.assertEquals(response.getStatusCode(), 200);\n
    Assert.assertEquals(response.jsonPath().getLong(id), petId);\n
    Assert.assertEquals(response.jsonPath().getString(name), Rex);\n
    Assert.assertEquals(response.jsonPath().getString(status), available);\n    }\n\n
    @Test(dependsOnMethods = {createPet})\n    public void getPetById() {\n        Response
    response = RestAssured\n        .given()\n        .when()\n        .get(/pet/{petId}),
    petId)\n        .then()\n        .extract()\n        .response();\n\n
    Assert.assertEquals(response.getStatusCode(), 200);\n
    Assert.assertEquals(response.jsonPath().getLong(id), petId);\n
    Assert.assertEquals(response.jsonPath().getString(name), Rex);\n
    Assert.assertEquals(response.jsonPath().getString(status), available);\n    }\n\n
    @Test(dependsOnMethods = {getPetById})\n    public void updatePet() {\n        Map<String,
    Object> updatedPetBody = new HashMap<>();\n        updatedPetBody.put(id, petId);\n
    updatedPetBody.put(category, Map.of(id, 1, name, Dogs));\n
    updatedPetBody.put(name, Max);\n        updatedPetBody.put(photoUrls,
    List.of(http://example.com/photo2.jpg));\n        updatedPetBody.put(tags,
    List.of(Map.of(id, 1, name, playful))); \n        updatedPetBody.put(status,
    sold);\n\n        Response response = RestAssured\n            .given()\n
    .contentType(ContentType.JSON)\n            .body(updatedPetBody)\n            .when()\n
    .put(/pet)\n            .then()\n            .extract()\n            .response();\n\n
    Assert.assertEquals(response.getStatusCode(), 200);\n
    Assert.assertEquals(response.jsonPath().getLong(id), petId);\n
    Assert.assertEquals(response.jsonPath().getString(name), Max);\n
    Assert.assertEquals(response.jsonPath().getString(status), sold);\n    }\n\n
    @Test(dependsOnMethods = {updatePet})\n    public void deletePet() {\n        Response
    deleteResponse = RestAssured\n        .given()\n        .when()\n
    .delete(/pet/{petId}), petId)\n        .then()\n        .extract()\n
    .response();\n\n        Assert.assertEquals(deleteResponse.getStatusCode(), 200);\n
    Assert.assertEquals(deleteResponse.jsonPath().getString(message),
    String.valueOf(petId));\n\n        Response getAfterDelete = RestAssured\n            .given()\n

```

```

.when()\n        .get("/pet/{petId}", petId)\n        .then()\n        .extract()\n        .response();\n\n    Assert.assertEquals(getAfterDelete.getStatusCode(), 404);\n    }\n}",

```

"reasoning": "The user wants Java Rest Assured code with TestNG, covering CRUD for Pet resource from Swagger Petstore. Must include imports, reusable base URI, @BeforeClass setup. Provide a test class with methods: createPet, getPetById, updatePet, deletePet. Must create request bodies, do assertions for status codes and important fields. Use meaningful names.\n\nNeed to manage petId across methods, probably static variable.\n\nUse RestAssured.baseURI = \"https://petstore.swagger.io/v2\"; (or maybe https://petstore.swagger.io/v2). Endpoints: POST /pet, GET /pet/{petId}, PUT /pet, DELETE /pet/{petId}.\n\nRequest body: JSON with fields id, name, category, photoUrls, tags, status. For simplicity, can use a Map or a POJO, but using JSONObject string.\n\nWe'll generate code.\n\nNeed imports: io.restassured.RestAssured; io.restassured.http.ContentType; io.restassured.response.Response; org.testng.annotations.*; org.testng.Assert; java.util.*;\n\nWe'll store petId as long.\n\nIn createPet test, generate random ID: long petId = System.currentTimeMillis(); store to static variable.\n\nCreate JSON body via a Map, then given().contentType(ContentType.JSON).body(map).post(\"/pet\")...\n\nAssertions: status code 200, response body \"id\" equals petId, \"name\" equals \"Rex\", \"status\" equals \"available\".\n\ngetPetById: GET /pet/{petId} and assert.\n\nupdatePet: modify name and status, use PUT /pet, body includes same id, new name, etc. Assert.\n\ndeletePet: DELETE /pet/{petId}, assert status code 200, then optionally GET and expect 404.\n\nMake methods dependOnMethods to maintain order.\n\nTest class name: PetApiCrudTest.\n\nWill include @BeforeClass to set baseURI.\n\nImplement.\n\nMake sure to import static methods for RestAssured.given maybe.\n\nLet's write code.\n\n"

```

    },
    "logprobs": null,
    "finish_reason": "stop"
}
],
"usage": {
    "queue_time": 0.11046104,
    "prompt_tokens": 396,
    "prompt_time": 0.018263247,
    "completion_tokens": 1367,

```

```
"completion_time": 2.859327072,
"total_tokens": 1763,
"total_time": 2.877590319,
"completion_tokens_details": {
  "reasoning_tokens": 417
}
},
"usage_breakdown": null,
"system_fingerprint": "fp_5082008e34",
"x_groq": {
  "id": "req_01kvgr6sjxeekvshh81132nky7",
  "seed": 1766251563
},
"service_tier": "on_demand"
}
```